PLATEFORME DE TRAITEMENT DES PAIEMENTS

STRATEGIE INTEGRATION DU ML



Nature du document	
Version	**1.0**
Client	**Étude de cas – STRIPE**
Date de la version	**09/10/2026**
Description de la version	**Bloc 2 - certification RNCP41993**
Modules concernés	**Livrable 5**
Auteur	**Ronan GUILLOU**

Table des matières

1 Concepts clés 3
1.1 Définitions 3
1.2 Principes 3
2 Vue d'ensemble 3
2.1 Le processus d'intégration 3
2.2 Les trois cas d'usage 3
2.3 Place du NoSQL 4
2.4 Schéma d'ensemble 4
3 Extraction des variables 4
3.1 Variables chaudes 4
3.2 Variables froides 4
4 Entrainement et Déploiement 4
4.1 Entraînement 5
4.2 Déploiement 5
5 Suivi de la performance 5
5.1 Ce qui est suivi 5
5.2 Alertes et décision 5
6 Conformité 5
5.3 Données personnelles dans les modèles 5
5.4 Explication des décisions 5
5.5 Responsabilités 5

Note sur l'usage de l'intelligence artificielle

Pour rédiger ce document, j'ai utilisé un assistant d'IA générative comme outil d'aide à la rédaction. Il est intervenu sur quatre points : la correction de l'orthographe et de la syntaxe, la proposition de structures et de brouillons de texte que j'ai relus et modifiés, la comparaison d'idées et d'options avant décision, et la vérification de la cohérence d'ensemble.

Les choix retenus, la sélection des besoins et la validation finale de chaque chapitre relèvent de ma responsabilité.

1 Concepts clés

Ce document décrit comment le **machine learning** s'intègre à la plateforme : les variables dont les modèles ont besoin, les trois cas d'usage (fraude, personnalisation, analyses prédictives), le déploiement des modèles, leur mise à jour et leur suivi. Il s'appuie sur les systèmes décrits dans les livrables précédents (OLTP Aurora, lakehouse Databricks, NoSQL DocumentDB, cache Redis).

1.1 Définitions

Terme	Définition
Feature	Information calculée à partir des données et donnée au modèle pour qu'il décide.
Variable chaude	Variable récente, mise à jour en quelques secondes et lue instantanément dans Redis.
Variable froide	Variable calculée par lots sur l'historique dans Gold, publiée chaque jour.
Entraînement	Phase où le modèle apprend à partir de l'historique des données.
Inférence	Utilisation du modèle entraîné pour donner un résultat sur un nouveau cas (par exemple scorer un paiement).
Registre de modèles	Catalogue qui conserve chaque version d'un modèle, son statut et son historique (MLflow).
Dérive	Baisse de qualité du modèle quand les données réelles s'éloignent de celles de l'entraînement.
Model Owner	Data scientist qui répond du modèle sur tout son cycle de vie (rôle défini dans la gouvernance du Bloc 1).

1.2 Principes

- **La décision de fraude se prend en moins de 100 ms.** Le modèle lit ses variables dans Redis, sans interroger les bases.
- **Le NoSQL fournit les signaux, les modèles vivent ailleurs.** DocumentDB et Redis alimentent les modèles en variables. Les modèles sont entraînés et versionnés dans Databricks (MLflow), puis servis soit par ECS Fargate dans des conteneurs (fraude), soit par lots dans Databricks (personnalisation, analyses prédictives).
- **Chaque décision est traçable.** Le score, la décision, les motifs et la version du modèle sont enregistrés avec le paiement.
- **Rien n'est mis en production sans validation.** Un nouveau modèle est comparé au modèle en place avant d'être publié, et un retour arrière reste possible.
- Chaque cas d'usage reçoit la fraîcheur que son usage exige, pas plus. Instantanée pour la fraude, quotidienne pour le reste.

2 Vue d'ensemble

Le machine learning s'intègre à la plateforme selon un processus en quatre étapes, commun aux trois cas d'usage.

2.1 Le processus d'intégration

- **Extraire les variables.** Les signaux bruts (paiements, sessions, clics) sont transformés en variables : chaudes dans Redis, froides dans Gold via le features engineering.
- **Entraîner.** Databricks entraîne les modèles à partir de Gold et enregistre chaque version dans MLflow.
- **Déployer.** Après validation, le modèle est mis en service : sur ECS pour la fraude, par lots dans Databricks pour les autres cas.
- **Suivre et mettre à jour.** La qualité des modèles est surveillée. Si elle baisse, le modèle est réentraîné et remplacé (boucle).

2.2 Les trois cas d'usage

Cas d'usage	Modèle	Exécution	Fraîcheur	Stratégie
Fraude en temps réel	XGBoost	ECS, temps réel	Instantanée	Score lu sur Redis, décision enregistrée avec le paiement, cas suspects envoyés à l'équipe fraude
Personnalisation	Filtrage collaboratif	Lot Databricks, servi via Redis	Calcul quotidien, lecture instantanée	Produits populaires pour les nouveaux clients

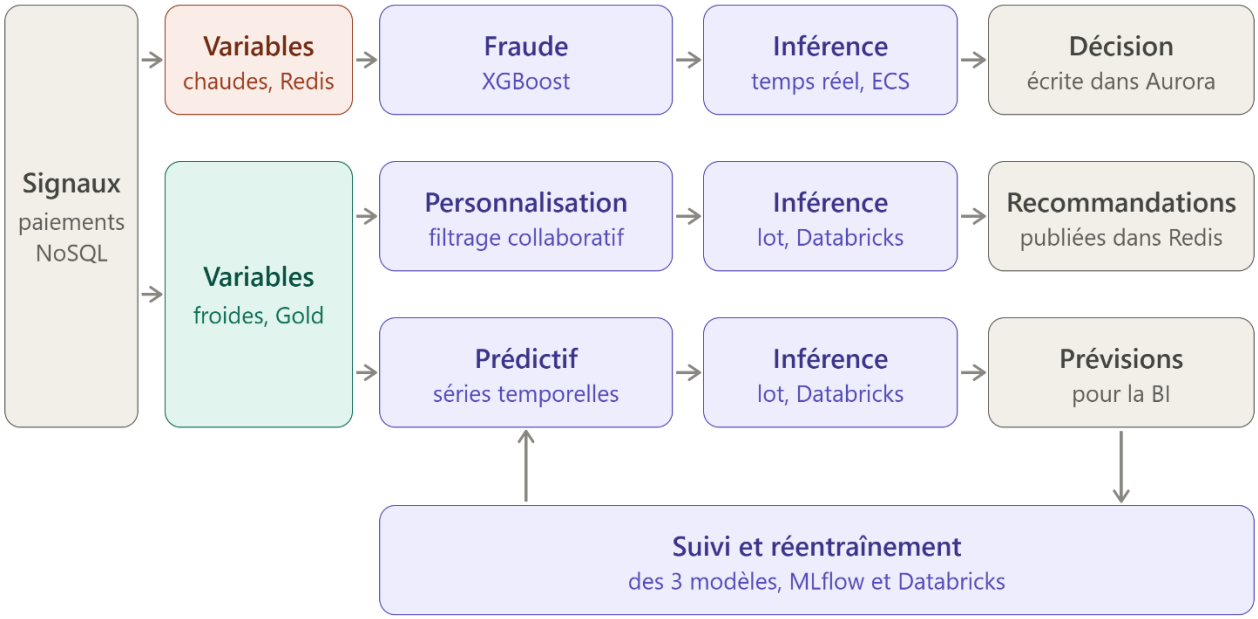
Analyses prédictives	Prévision sur séries temporelles	Lot Databricks	Quotidienne	Résultats rendus à la BI et à la direction financière

Les trois cas ne partagent pas le même modèle, car ils ne posent pas la même question. Ils partagent en revanche la même plateforme : mêmes données sources (NoSQL et Gold), même registre MLflow, même supervision, même gouvernance (Model Owner, Unity Catalog).

2.3 Place du NoSQL

Dans notre architecture, le NoSQL joue le rôle de **fournisseur de signaux** : les collections sessions, click_events et ml_features alimentent les variables des modèles, en direct via Redis (compteur C8) et par lots via Bronze, Silver et Gold.
Les modèles eux-mêmes sont entraînés et servis hors de DocumentDB, ce qui protège le chemin de paiement et garde chaque brique dans son rôle.

2.4 Schéma d'ensemble



3 Extraction des variables

Elles se répartissent en deux familles selon leur fraîcheur. Le détail des compteurs est dans le Livrable 3 et celui des flux dans le Livrable 4.

3.1 Variables chaudes

Ce sont les variables sur le comportement récent : les compteurs de fraude (paiements, cartes, montants, adresses IP, marchands, profil de session). Elles sont mises à jour en continu dans Redis et lues par le scoring en moins d'une milliseconde, sans interroger les bases. Elles servent à la fraude.

3.2 Variables froides

Ce sont les variables sur le comportement de fond, calculées par lots dans Gold (par exemple l'historique d'un client sur 90 jours). Elles servent aux trois cas d'usage :

- **Fraude** : publiées chaque jour dans Redis, à côté des variables chaudes.
- Personnalisation et analyses prédictives : lues directement dans Gold.
- **Entraînement** : les modèles apprennent sur les mêmes variables que celles utilisées en production.

4 Entrainement et Déploiement

Le processus est le même pour les trois modèles.

4.1 Entraînement

Les modèles sont entraînés dans Databricks à partir de Gold. Chaque entraînement est enregistré dans MLflow (version du modèle, résultats, données utilisées). L'origine des données est documentée par le lignage d'Unity Catalog et les versions des tables Delta.

4.2 Déploiement

1. **Enregistrement** : chaque modèle entraîné devient une nouvelle version « challenger » dans le registre MLflow.
2. **Validation** : la nouvelle version est comparée au modèle en place, et le Model Owner valide avant toute mise en ligne. Le modèle est manuellement étiqueté « production » s'il est validé.
3. **Mise en service** : sur AWS ECS (Fargate) pour la fraude (dans un conteneur), par lots Databricks pour la personnalisation et le prédictif.
4. **Retour arrière** : l'ancienne version reste disponible dans le registre.

Chaque décision de fraude enregistre la version du modèle qui l'a prise dans fraud_assessment.

5 Suivi de la performance

Un modèle se dégrade avec le temps. Le suivi sert à le voir tôt et à déclencher un réentraînement (chapitre 4). Il porte sur deux choses : la qualité du modèle et le bon fonctionnement du service.

5.1 Ce qui est suivi

- **Qualité du modèle** (Databricks : MLflow, Lakehouse Monitoring) : fraudes détectées, faux refus, et dérive des données, c'est-à-dire quand les variables s'éloignent de celles de l'entraînement.
- **Service de scoring** (CloudWatch) : temps de réponse (moins de 100 ms), erreurs et disponibilité.
- **Personnalisation et prédictif** (Databricks) : à chaque lot, pertinence des recommandations et écart entre prévisions et réalité.

5.2 Alertes et décision

Les indicateurs sont affichés dans un tableau de bord alimenté par Gold. Si un seuil est dépassé, une alerte part vers l'équipe d'exploitation (CloudWatch). Le Model Owner analyse, puis décide d'un réentraînement ou d'un retour à la version précédente.

6 Conformité

La conformité (RGPD, CCPA, PCI-DSS) est traitée dans le Livrable 6. Ce chapitre ne reprend que ce qui concerne les modèles

5.3 Données personnelles dans les modèles

Les variables utilisent des identifiants pseudonymisés (clés Redis pseudonymisées).

Quand une personne demande l'effacement de ses données, elles sont effacées dans toutes les couches (Bronze, Silver, Gold). Les modèles suivants sont donc entraînés sans ces données, dès le prochain réentraînement hebdomadaire.

5.4 Explication des décisions

Chaque décision de fraude est enregistrée avec son score, ses motifs et la version du modèle dans la table fraud_assessment. On peut ainsi expliquer pourquoi un paiement a été refusé, et l'équipe fraude peut revoir les cas suspects.

5.5 Responsabilités

Model Owner : répond du modèle sur tout son cycle de vie et valide les mises en ligne.
Équipe fraude : traite les paiements suspects.
DPO : veille à la protection des données personnelles utilisées par les modèles.